13 · Profile

Personal details, profile and cover images, and changing the email address.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The **Profile** page edits name, phone, bio, location and date of birth (plus department for workers), uploads or removes a profile picture and a cover image, and changes the email address. A new email only takes effect after the link sent to it is clicked; the current one keeps working until then.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/profile.py     GET/PATCH /profile, POST/DELETE /profile/avatar and /profile/cover,
│                             POST /profile/email, POST /profile/email/confirm
├── main.py                   serves uploaded images at /api/v1/media
└── services/mailer.py        sends the confirmation email
frontend/src/
├── pages/Profile.tsx         the page and ImagePicker
├── pages/ConfirmEmail.tsx    opened from the email link
└── services/account.ts       getProfile(), updateProfile(), uploadImage(), removeImage(),
                              requestEmailChange()
```

How the pieces connect

1. Image: `ImagePicker` opens the file dialog, checks the size, and sends the file as form data.
2. The backend reads at most the size limit plus one byte, checks the file's real type from its first bytes, saves it under a random name and deletes the old one.
3. Email: the backend checks the password and that the address is free, stores a hashed one-time token and emails the link.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/profile.py` · _sniff

```
141  def _sniff(head: bytes) -> str | None:
142      for signature, ext in SIGNATURES:
143          if head.startswith(signature):
144              return ext
145      if head[:4] == b"RIFF" and head[8:12] == b"WEBP":
146          return "webp"
147      return None
```

What the code means

- Image formats start with known bytes (a **signature**): PNG starts with `\x89PNG`, JPEG with `\xff\xd8\xff`, WebP has `RIFF....WEBP`.
- Checking the bytes, not the file name, means a renamed file of another type is refused.
- `b"..."` is a **bytes** literal; `head[8:12]` takes bytes 8 to 11.

2. `backend/app/api/routes/profile.py` · _store

```
161 async def _store(file: UploadFile, kind: Kind, user: User) -> str:
162     limit = MAX_BYTES[kind]
163     content = await file.read(limit + 1)
164     if len(content) > limit:
165         raise HTTPException(status.HTTP_413_REQUEST_ENTITY_TOO_LARGE, f"Images must be {limit // (1024 * 102
                4)} MB or smaller.")
166     ext = _sniff(content[:16])
167     if ext is None:
168         raise HTTPException(status.HTTP_415_UNSUPPORTED_MEDIA_TYPE, "Upload a PNG, JPEG or WebP image.")
169     folder = media_root() / kind
170     folder.mkdir(parents=True, exist_ok=True)
171     # Unguessable name, so a URL cannot be derived from a user id.
172     name = f"{secrets.token_urlsafe(18)}.{ext}"
173     (folder / name).write_bytes(content)
174     _remove_old(user.avatar_url if kind == "avatars" else user.cover_url, kind)
175     return f"{MEDIA_URL}/{kind}/{name}"
```

What the code means

- `await file.read(limit + 1)`: reading one byte past the limit is enough to know the file is too big, without reading all of it.
- 413 (too large) and 415 (wrong type) are the standard HTTP codes for these cases.
- `secrets.token_urlsafe(18)` makes a random, unguessable file name, so nobody can work out a person's image URL.
- `_remove_old(...)` deletes the previous image, only ever inside the media folder.

3. `backend/app/api/routes/profile.py` · upload_avatar

```
178 @router.post("/avatar", response_model=ProfileOut)
179 async def upload_avatar(
180     file: UploadFile = File(...), user: User = Depends(require_any_verified), db: AsyncSession = Depends(get
            _session)
181 ) -> ProfileOut:
182     user.avatar_url = await _store(file, "avatars", user)
183     await db.commit()
184     return await _profile_out(db, user)
```

What the code means

- `file: UploadFile = File(...)` reads a file from a multipart form upload; `...` means it is required.

4. `backend/app/api/routes/profile.py` · request_email_change

```
223 @router.post("/email", response_model=Message)
224 async def request_email_change(
225     data: EmailChange,
226     background: BackgroundTasks,
227     user: User = Depends(require_any_verified),
228     db: AsyncSession = Depends(get_session),
229 ) -> Message:
230     if not verify_password(data.password, user.hashed_password):
231         raise HTTPException(status.HTTP_400_BAD_REQUEST, "Your password is not correct.")
232     if data.new_email == user.email:
233         raise HTTPException(status.HTTP_400_BAD_REQUEST, "That is already your email address.")
234     if await db.scalar(select(User.id).where(User.email == data.new_email)):
235         raise HTTPException(status.HTTP_409_CONFLICT, EMAIL_TAKEN)
236     now = utcnow()
237     sent_at = user.email_token_expires_at - VERIFICATION_TOKEN_TTL if user.email_token_expires_at else None
238     if sent_at and user.pending_email == data.new_email and now - sent_at < EMAIL_RESEND_INTERVAL:
239         raise HTTPException(status.HTTP_429_TOO_MANY_REQUESTS, "Please wait a minute before asking for anoth
                er email.")
240     raw = new_token()
241     user.pending_email = data.new_email
242     user.email_token_hash = hash_token(raw)
243     user.email_token_expires_at = now + VERIFICATION_TOKEN_TTL
244     await db.commit()
245     background.add_task(send_email_change_email, data.new_email, user.name, raw)
246     return Message(detail=f"We sent a confirmation link to {data.new_email}. Your current address works unti
            l you confirm.")
```

What the code means

- The current password is required, so someone at an unlocked computer cannot redirect the account.
- Only the **hash** of the token is stored (`hash_token(raw)`); the raw token travels only in the email link.
- `background.add_task(...)` sends the email after the response, so the page does not wait for the mail server.

5. `frontend/src/pages/Profile.tsx` · ImagePicker

```
13  function ImagePicker({ kind, onDone, label }: { kind: "avatar" | "cover"; onDone: (p: ProfileData) => void;
        label: string }) {
14    const input = useRef<HTMLInputElement>(null);
15    const [busy, setBusy] = useState(false);
16    const [error, setError] = useState<string | null>(null);
17    const limitMb = kind === "avatar" ? 2 : 5;
18
19    const pick = async (file: File | undefined) => {
20      if (!file) return;
21      setError(null);
22      if (file.size > limitMb * 1024 * 1024) {
23        setError(`Images must be ${limitMb} MB or smaller.`);
24        return;
25      }
26      setBusy(true);
27      try {
28        onDone(await account.uploadImage(kind, file));
29      } catch (e) {
30        setError(errorMessage(e));
31      } finally {
32        setBusy(false);
33        if (input.current) input.current.value = "";
34      }
35    };
36    return (
37      <span className="inline-flex flex-col gap-1">
38        <input ref={input} type="file" accept={ACCEPT} hidden onChange={(e) => void pick(e.target.files?.[0])}
              />
    ... (7 more lines in the file: the hidden input and button follow)
```

What the code means

- `useRef<HTMLInputElement>(null)` points at a hidden file input; the visible button calls `input.current?.click()` to open the file dialog.
- The size is checked in the browser first, to save a pointless upload.
- `finally` clears the input's value so choosing the same file again still triggers `onChange`.

6. `frontend/src/services/account.ts` · uploadImage

```
122  export function uploadImage(kind: "avatar" | "cover", file: File) {
123    const form = new FormData();
124    form.append("file", file);
125    return data(api.post<Profile>(`/profile/${kind}`, form));
126  }
```

What the code means

- `FormData` builds a multipart upload; the field name `file` matches `file: UploadFile` on the backend.

Syntax used in this feature

Code	What it means
`b"\x89PNG"`	A bytes literal (raw file bytes).
`UploadFile = File(...)`	A required uploaded file.
`BackgroundTasks.add_task(f, ...)`	Run f after the response is sent.
`new FormData(); form.append(k, v)`	A form upload built in code.